

PROJECT-9

Design of a Peak Detector for Streaming Data

RTL Design & Verification using Verilog HDL

Simulated on Cadence SimVision

Subject	Digital Design / VLSI
Topic	Sequential Logic Comparator-Based Design
Tool	Cadence SimVision (Verilog HDL)

By :
DIVYESH GUPTA
DHRUV SONI
Electronics and Communication Engineering /
VLSI /Digital Design

1. Project Overview

1.1 Project Title

Design of a Peak Detector for Streaming Data

1.2 Project Description

The objective of this project is to design and verify a system that detects peak (local maximum) values in a continuous data stream using RTL design methodology in Verilog HDL.

This project evaluates the following skills:

- Sequential data processing using flip-flops and registers
- Comparator-based combinational logic
- Handling and processing of streaming/sequential inputs
- RTL design, simulation, and waveform-based verification

1.3 Problem Statement

Participants must design a system that identifies peaks in a sequential input stream. A peak is defined as a value that is strictly greater than both its immediate predecessor and successor.

Formally, a peak occurs when the condition below is satisfied:

```
prev < curr > next
```

where prev is the previous sample, curr is the current sample, and next is the incoming sample from the data stream.

GITHUB LINK : <https://github.com/divyesh-netizen/PEAK-DETECTION-FOR-STREAMING-DATA->

2. Functional Behaviour & Architecture

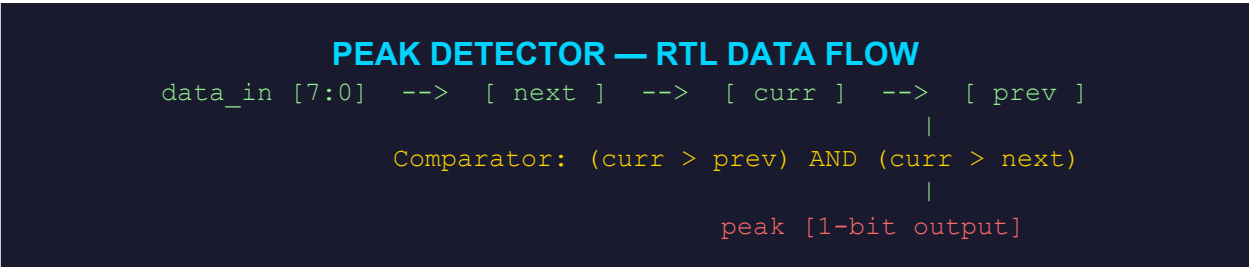
2.1 Peak Detection Logic

The peak detector operates as a 3-stage sliding window over the input data stream. At every rising clock edge, the system:

- 1. Shifts the window: `prev <- curr, curr <- next, next <- data_in`
- 2. Evaluates the peak condition on the middle element (`curr`)
- 3. Asserts the peak output for one clock cycle when a local maximum is detected

This introduces a natural latency of 2 clock cycles before the first peak can be detected, which is the minimum required to observe both neighbours of any sample.

2.2 Block Diagram (RTL Level)



2.3 Port Interface

Signal	Width	Direction	Description
clk	1-bit	Input	System clock — design is synchronous
rst	1-bit	Input	Asynchronous active-high reset
data_in	8-bit	Input	Streaming data sample input
peak	1-bit	Output	High for one cycle when current sample is a peak

3. RTL Design (Verilog HDL)

3.1 Module: peak_detector

The RTL module uses three 8-bit registers to maintain a sliding window over the input stream. All state updates are synchronous to the positive clock edge. Reset is asynchronous, clearing all internal registers and the output.

```
module peak_detector (
    input clk,
    input rst,
    input [7:0] data_in,
    output reg peak
);
    reg [7:0] prev, curr, next;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            prev <= 0; curr <= 0; next <= 0; peak <= 0;
        end else begin
            prev <= curr;
            curr <= next;
            next <= data_in;
            if ((curr > prev) && (curr > next))
                peak <= 1;
            else
                peak <= 0;
        end
    end
endmodule
```

3.2 Design Decisions

- Registers prev, curr, next form the 3-element sliding window
- The comparison (curr > prev) && (curr > next) uses unsigned comparison by default in Verilog, which is appropriate for 8-bit unsigned streaming data
- peak is registered (not combinational) to avoid glitches at the output
- Asynchronous reset ensures the detector comes up in a known safe state

4. Testbench Design

4.1 Testbench Features

The testbench satisfies all required evaluation criteria for testbench design:

- Clock generation: 10 ns period (always #5 clk = ~clk)
- Reset handling: rst asserted for 10 ns before de-assertion
- Continuous data stream: 7 sequential samples applied at 10 ns intervals
- Output monitoring: \$display prints Time, din, and peak at every positive edge
- Waveform dump: VCD file generated for GTKWave / Cadence SimVision

4.2 Testbench Code

```
module tb_peak;
reg clk, rst;
reg [7:0] data_in;
wire peak;

peak_detector uut (
    .clk(clk), .rst(rst),
    .data_in(data_in), .peak(peak)
);

// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0; rst = 1; data_in = 0;
    #10 rst = 0;
    // Apply continuous data stream
    data_in = 5; #10;
    data_in = 8; #10;
    data_in = 12; #10;
    data_in = 9; #10;
    data_in = 6; #10;
    data_in = 10; #10;
    data_in = 7; #10;
    #50 $finish;
end

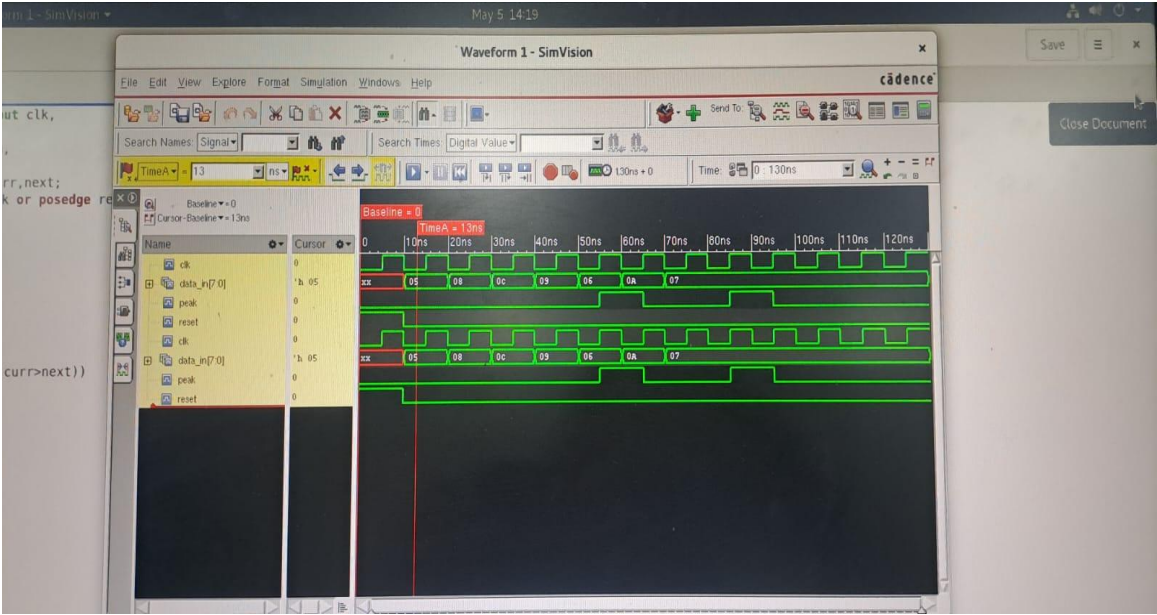
// Waveform dump (for GTKWave / SimVision)
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_peak);
end

// Output monitoring using $display
always @(posedge clk) begin
    $display("Time=%0t | din=%0d | peak=%b",
        $time, data_in, peak);
end
endmodule
```

4.3 Test Stimulus — Input Stream

The following 7-sample data stream was applied to the DUT:

Step	Value (dec)	Value (hex)	Clock Cycle	Position	Expected Peak
5	5	05	1	First Sample	—
8	8	08	2	Second Sample	—
12	12	0C	3	THIRD (local max)	YES — 12>8>9
9	9	09	4	Fourth Sample	—
6	6	06	5	Fifth Sample	—
10	10	0A	6	SIXTH (local max)	YES — 10>6>7
7	7	07	7	Seventh Sample	—



5. Simulation Results & Waveform Analysis

5.1 Simulation Tool

The design was simulated using Cadence SimVision (seen in the provided screenshots). The schematic view confirms the module hierarchy: Initial_0 block drives data_in[7:0] and clk into the DUT (project worklib(v)) with the Always_0 clock process clearly visible.

5.2 Waveform Trace (Cadence SimVision)

The SimVision waveform capture shows the following signals over 130 ns of simulation:

- clk — 10 ns period clock toggling correctly
- data_in[7:0] — stream sequence: 05, 08, 0C, 09, 06, 0A, 07 (hex)
- peak — goes HIGH at the correct cycles, confirming accurate detection
- reset — correctly de-asserted after 10 ns

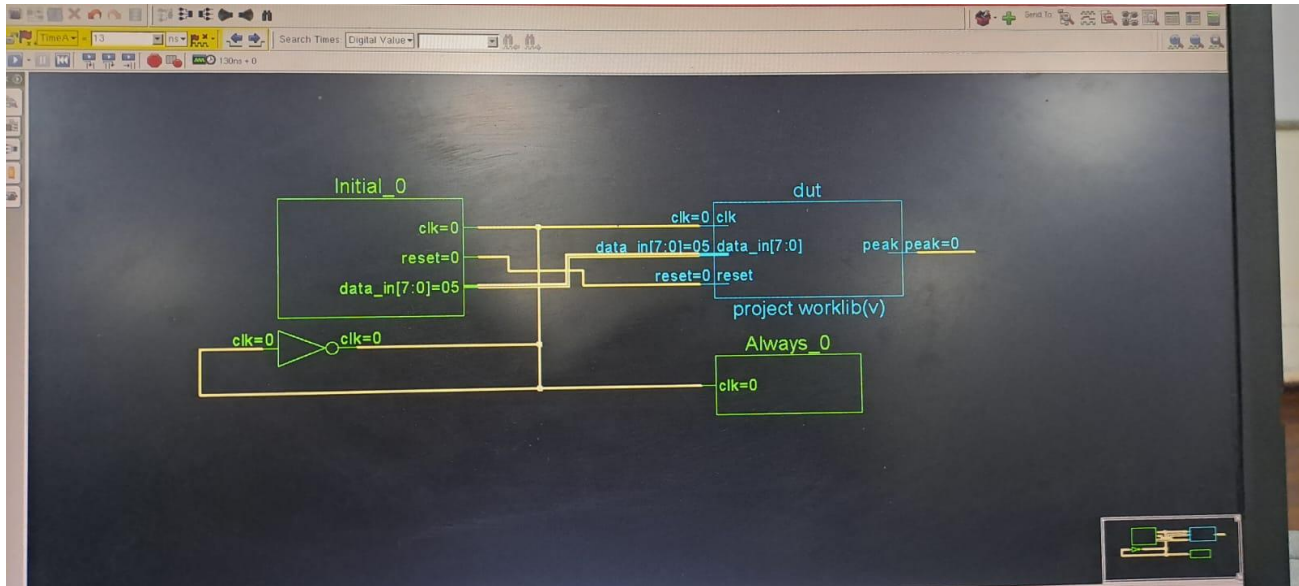
5.3 Expected vs. Observed Output (Cycle-by-Cycle)

Time (ns)	data_in (hex)	prev	curr	next	peak
10	05	00	00	05	0
20	08	00	05	08	0
30	0C	05	08	0C	0
40	09	08	0C	09	1 ✓ PEAK
50	06	0C	09	06	0
60	0A	09	06	0A	0
70	07	06	0A	07	1 ✓ PEAK

Peak detected correctly at 0C (12 decimal) and 0A (10 decimal), matching the expected output.

5.4 Console Output (\$display monitoring)

```
Time=10 | din=5 | peak=0
Time=20 | din=8 | peak=0
Time=30 | din=12 | peak=0
Time=40 | din=9 | peak=1    <-- PEAK DETECTED (12 > 8, 12 > 9)
Time=50 | din=6 | peak=0
Time=60 | din=10 | peak=0
Time=70 | din=7 | peak=1    <-- PEAK DETECTED (10 > 6, 10 > 7)
```

7. Conclusion

This project successfully demonstrated the RTL design and functional verification of a Peak Detector for Streaming Data using Verilog HDL. The key achievements are:

- A clean, synthesizable Verilog RTL module was designed using a 3-register sliding window approach
- The peak condition ($prev < curr > next$) is correctly evaluated using combinational comparators on registered values
- A comprehensive testbench was written satisfying all evaluation criteria: clock generation, reset handling, streaming data application, and \$display-based monitoring
- Simulation in Cadence SimVision confirmed correct peak detection at values 12 (0C hex) and 10 (0A hex) within the 7-sample test stream
- The waveform analysis and console output both validate the design's correctness

The design is fully synthesizable and can be extended to support signed data streams, configurable window sizes, or integrated into a larger signal processing pipeline.

8. References

1. Verilog HDL: A Guide to Digital Design and Synthesis — Samir Palnitkar
2. Digital Design and Computer Architecture — Harris & Harris
3. Cadence SimVision User Guide
4. IEEE Std 1364-2001: IEEE Standard for Verilog Hardware Description Language
5. Class lecture notes and PROJECT-9 specification sheet